

1. Introduction

1.1 Project Overview

EduMind AI is a full-stack, AI-powered academic advisory platform designed specifically for university students in Bangladesh. The system integrates large language models (LLMs) and intelligent automation into a single cohesive web application accessible via any modern browser.

The platform addresses the core pain points that Bangladeshi university students face daily: disorganized study schedules, lack of personalized academic assistance, inefficient self-assessment, and language barriers in accessing digital study tools. EduMind AI is built with a modern technology stack — React.js on the frontend, Node.js/Express on the backend, PostgreSQL for persistence, and the OpenAI API (GPT-4o-mini) as the AI engine.

1.2 Objectives

The primary objectives of this system are:

1. Build a secure, full-stack web application with JWT-based authentication and role-based access control (RBAC).
2. Develop a real-time AI chatbot with full conversation memory that answers academic questions with contextual understanding.
3. Implement an AI quiz generator that creates practice questions from student-uploaded PDF or text-based notes.
4. Create an AI study planner that generates personalized weekly schedules based on course load, available hours, and deadlines.
5. Build a smart resource recommendation engine that suggests relevant videos, articles, and practice problems.
6. Implement an AI-powered performance analytics dashboard with visual charts and automatic weak-area detection.
7. Add bilingual AI support (English + Bangla) so students can interact in their preferred language.
8. Build a Pomodoro-integrated focus timer with AI-suggested session lengths based on topic complexity.
9. Deploy the complete application to a live, production-ready URL with proper environment variable management.
10. Maintain full technical documentation, a well-organized GitHub repository, and weekly progress commits throughout development.

1.3 Scope

1.3.1 In-Scope

The following functional areas are within the scope of this project:

- User authentication and authorization (register, login, logout, JWT, bcrypt)
- Student dashboard with study streaks, task overview, and AI-generated tips
- Task and deadline tracker with countdown timers
- Note management system supporting PDF and plain-text uploads
- Admin panel for user management, activity logs, and platform statistics
- Fully responsive UI (desktop, tablet, and mobile)
- AI Chatbot with contextual conversation memory
- AI Quiz Generator from uploaded notes (MCQ + short answer)
- AI Study Planner with weekly timetable generation
- AI Resource Recommender (YouTube, articles, practice sites)
- AI Weak Area Detector from quiz history
- Bilingual support (English and Bangla toggle)
- AI Flashcard Generator with spaced repetition
- Pomodoro Focus Timer with AI-recommended session durations
- Exam Readiness Score (0-100) with actionable advice
- Visual analytics dashboard (study hours, quiz trends, topic coverage)

1.3.2 Out-of-Scope

The following are explicitly excluded from this version of the system:

- Native mobile applications (Android or iOS)
- Real-time collaborative study rooms or multiplayer features
- Payment or subscription management
- Integration with university LMS platforms (Moodle, Google Classroom)
- Video streaming or live tutoring functionality

2. Functional Requirements

2.1 User Authentication & Authorization

- FR-001: A new user shall be able to register by providing a name, email address, and password. The system shall hash the password using bcrypt before storing it.
- FR-002: A registered user shall be able to log in using their email and password. The system shall issue a signed JWT access token upon successful authentication.
- FR-003: An authenticated user shall be able to log out. The system shall invalidate the active session token.
- FR-004: The system shall enforce role-based access control (RBAC) distinguishing between Student and Admin roles.
- FR-005: Protected routes shall reject requests with missing, expired, or invalid JWT tokens and return an appropriate error response.

2.2 Student Dashboard

- FR-006: The dashboard shall display the student's current study streak (consecutive active days).
- FR-007: The dashboard shall show a summary of upcoming deadlines and tasks sorted by proximity.
- FR-008: The dashboard shall display AI-generated daily study tips refreshed on each login.
- FR-009: The dashboard shall provide quick-access navigation to all major features (chatbot, quiz, planner, analytics).
- FR-010: The dashboard shall display the student's Exam Readiness Score with a visual indicator.

2.3 Task & Deadline Tracker

- FR-011: A student shall be able to add a task with a title, associated course, task type (assignment, exam, project), and due date.
- FR-012: Each task shall display a visual countdown timer showing days and hours remaining.
- FR-013: A student shall be able to mark tasks as complete and delete tasks.
- FR-014: The system shall visually differentiate overdue tasks from upcoming tasks.

2.4 Note Management

- FR-015: A student shall be able to upload study notes in PDF or plain-text (.txt) format.
- FR-016: Notes shall be organized by subject and retrievable via a subject-based filter.
- FR-017: The system shall extract and store text content from uploaded PDFs using pdf-parse.
- FR-018: A student shall be able to delete uploaded notes from their account.

2.5 AI Chatbot

- FR-019: The AI chatbot shall accept user questions about academic topics and return contextually accurate answers.
- FR-020: The chatbot shall maintain full conversation history within a session and persist context across sessions for each user.
- FR-021: Chatbot responses shall be streamed to the client in real-time using Server-Sent Events (SSE).
- FR-022: The chatbot shall support Markdown rendering of responses (code blocks, bullet points, headings).
- FR-023: The chatbot shall support a bilingual mode. When Bangla is selected, the system prompt shall instruct the AI to respond using Bangla academic vocabulary.

2.6 AI Quiz Generator

- FR-024: A student shall be able to select an uploaded note and trigger quiz generation.
- FR-025: The system shall extract note text and send it to the AI with a structured JSON prompt to generate MCQs and short-answer questions.
- FR-026: Generated quizzes shall be rendered as interactive question cards with selectable answer choices.
- FR-027: Upon quiz completion, the system shall display a score summary and store the result in the database.
- FR-028: Quiz scores and topic history shall be used by the Weak Area Detector.

2.7 AI Study Planner

- FR-029: A student shall be able to input course names, daily available study hours, and exam dates.
- FR-030: The AI shall generate a structured weekly timetable prioritizing high-risk subjects (those with nearest deadlines or lowest quiz scores).
- FR-031: The generated study plan shall be displayed in a weekly grid view and be downloadable or printable.

2.8 AI Resource Recommender

- FR-032: The system shall analyze the student's current chatbot topic and quiz history to determine active study areas.
- FR-033: The AI shall recommend specific YouTube channels, articles, and practice problem sites relevant to the detected topic.
- FR-034: Each recommendation shall include a brief explanation of relevance.

2.9 AI Weak Area Detector

- FR-035: The system shall analyze quiz score history to identify topics where the student consistently scores below a defined threshold (default: 60%).
- FR-036: Identified weak areas shall be surfaced on the student dashboard and analytics page.
- FR-037: The AI shall generate a short personalized study advice message for each detected weak area.

2.10 AI Flashcard Generator

- FR-038: A student shall be able to select an uploaded note to generate flashcards automatically.
- FR-039: Flashcards shall be rendered in a flip-card UI where the front shows a question and the back shows the answer.
- FR-040: Students shall be able to self-rate each card (Easy / Medium / Hard) to implement a basic spaced-repetition scheduling algorithm.

2.11 Pomodoro Focus Timer

- FR-041: The system shall provide a Pomodoro-style countdown timer with configurable work and break durations.
- FR-042: The AI shall recommend an optimal session length based on the topic the student is currently studying (e.g., 25 min for review, 50 min for new complex topics).
- FR-043: Completed focus sessions shall be logged and contribute to the student's study streak and analytics data.

2.12 Exam Readiness Score

- FR-044: The system shall compute a 0-100 exam readiness score for each course based on quiz performance, study hours logged, and weak area resolution rate.
- FR-045: The score shall be accompanied by AI-generated textual advice recommending specific actions to improve readiness.

2.13 Analytics Dashboard

- FR-046: The dashboard shall display line charts showing daily study hours over the past 30 days.
- FR-047: The dashboard shall display bar charts showing quiz performance trends by topic.
- FR-048: The dashboard shall display a topic coverage pie chart showing percentage of syllabus areas studied.
- FR-049: All chart data shall be auto-populated from logged activity with no manual student input required.

2.14 Admin Panel

- FR-050: An admin shall be able to view a list of all registered users with their registration date and last active timestamp.
- FR-051: An admin shall be able to view platform-wide statistics: total users, total quizzes generated, total study hours logged.
- FR-052: An admin shall be able to deactivate or delete a student account.
- FR-053: An admin shall be able to view an activity log showing recent user actions on the platform.

3. Non-Functional Requirements

3.1 Performance

- NFR-001: API response time for standard CRUD endpoints (tasks, notes, authentication) shall not exceed 500 ms under normal operating conditions.
- NFR-002: AI chatbot responses shall begin streaming within 3 seconds of the user submitting a query.
- NFR-003: The frontend shall achieve a Google Lighthouse performance score of at least 80 on desktop.
- NFR-004: The system shall support at least 50 concurrent active users without degradation in response time.

3.2 Security

- NFR-005: All passwords shall be hashed using bcrypt with a minimum work factor of 10 before storage. Plaintext passwords shall never be stored or logged.
- NFR-006: All API endpoints requiring authentication shall validate JWT tokens on every request. Expired or tampered tokens shall be rejected with HTTP 401.
- NFR-007: Sensitive configuration values (API keys, database credentials, JWT secrets) shall be stored in environment variables and never committed to the repository.
- NFR-008: The application shall be protected against common OWASP vulnerabilities including SQL injection, XSS, and CSRF.
- NFR-009: File uploads shall be restricted by MIME type and file size (maximum 10 MB per upload) to prevent malicious uploads.

3.3 Usability

- NFR-010: The user interface shall be fully responsive and render correctly on screen widths from 320 px (mobile) to 2560 px (4K desktop).
- NFR-011: The system shall provide clear, user-friendly error messages for all failed operations (e.g., invalid login, upload failure, AI timeout).
- NFR-012: Loading states and skeleton screens shall be displayed while asynchronous operations are in progress to avoid a blank-screen experience.
- NFR-013: AI-generated content (quiz questions, study plans, chatbot responses) shall be presented in readable Markdown with appropriate formatting.

3.4 Reliability & Availability

- NFR-014: The live deployment shall target 99% uptime during the academic semester, leveraging Vercel (frontend) and Render (backend) managed infrastructure.
- NFR-015: In the event of an AI API failure or timeout, the system shall display a graceful fallback message rather than an unhandled error.
- NFR-016: Database operations shall use transactions where multiple related writes occur to ensure data consistency.

3.5 Maintainability

- NFR-017: Source code shall follow a consistent style guide and be organized by feature modules (auth, notes, quiz, planner, etc.).
- NFR-018: The GitHub repository shall maintain a well-documented README, a consistent commit history (minimum one commit per development day), and a branching strategy separating frontend, backend, and AI integration work.
- NFR-019: All AI prompts shall be documented in a dedicated prompt engineering file within the repository for reproducibility and audit.

3.6 Scalability

- NFR-020: The backend architecture shall be stateless (JWT-based) so that additional server instances can be added horizontally without session affinity requirements.
- NFR-021: The database schema shall be normalized to at least Third Normal Form (3NF) to support future expansion without major restructuring.

4. User Roles

4.1 Role Overview

EduMind AI defines two primary user roles, each with a distinct set of permissions and access levels. Role assignment is made at registration and stored in the users table. All authorization checks are enforced server-side via JWT payload inspection.

Role	Description	Default Permissions
Student	The primary end-user of the platform. Enrolled university students who interact with all AI-powered academic features.	Access dashboard, chatbot, quiz, planner, flashcards, timer, analytics, and profile settings. Upload and manage personal notes.
Admin	Platform administrator responsible for user governance, monitoring, and platform health oversight.	All Student permissions + access to admin panel: view/deactivate users, view activity logs, view platform-wide statistics.

4.2 Student Role — Detailed Capabilities

- Register and authenticate with a personal account
- View and update their profile (avatar, course list, notification preferences)
- Add, view, and delete tasks and deadlines
- Upload, organize, and delete personal notes (PDF and text)
- Interact with the AI Chatbot in English or Bangla
- Generate and take AI-powered quizzes from their notes
- Generate and use AI-powered flashcards
- Request an AI-generated weekly study plan
- View AI-recommended study resources
- Use the Pomodoro Focus Timer with AI session recommendations
- View their personal analytics dashboard (study hours, quiz trends, weak areas)
- View their Exam Readiness Score per course

4.3 Admin Role — Detailed Capabilities

- All capabilities of the Student role
- Access the Admin Panel from the navigation menu
- View a paginated list of all registered users
- View per-user activity metadata (registration date, last active, total quizzes)
- Deactivate or permanently delete a student account
- View platform-wide aggregate statistics
- View a chronological activity log of all user actions on the platform

5. Use Cases

5.1 Use Case: User Registration

Field	Detail
Use Case ID	UC-001
Use Case Name	User Registration
Actor	Unregistered Visitor
Precondition	The user has navigated to the EduMind AI registration page.
Trigger	User submits the registration form.
Main Flow	1. User enters name, email, and password. 2. User clicks the Register button. 3. System validates all fields (non-empty, valid email format, password ≥ 8 characters). 4. System checks that the email does not already exist in the database. 5. System hashes the password using bcrypt. 6. System inserts a new user record into the database with the role Student. 7. System issues a JWT access token and returns it in the response. 8. User is redirected to the Student Dashboard.
Alternate Flow	A: If the email already exists, the system returns a 409 Conflict error and displays the message 'An account with this email already exists.'
Postcondition	A new student account is created. The user is authenticated and can access all student-role features.

5.2 Use Case: Student Login

Field	Detail
Use Case ID	UC-002
Use Case Name	Student Login
Actor	Registered Student
Precondition	The student has a registered account and is on the login page.
Trigger	User submits the login form.
Main Flow	1. User enters their registered email and password. 2. User clicks the Login button. 3. System looks up the user record by email. 4. System compares the submitted password against the stored bcrypt hash. 5. If valid, system generates a new JWT token with the user's ID and role. 6. Token is stored client-side (httpOnly cookie or localStorage). 7. User is redirected to the Student Dashboard.
Alternate Flow	A: If email is not found or password does not match, the system returns HTTP 401 and displays 'Invalid email or password.' No specific indication of which field is incorrect is given to prevent enumeration attacks.
Postcondition	The student is authenticated. Subsequent requests include the JWT token for authorization.

5.3 Use Case: AI Chatbot Interaction

Field	Detail
Use Case ID	UC-003
Use Case Name	AI Chatbot Interaction
Actor	Authenticated Student
Precondition	The student is logged in and has navigated to the Chatbot screen.
Trigger	Student submits a question or academic query in the chat input.
Main Flow	1. Student types a question and presses Send. 2. System appends the new message to the stored conversation history for this student. 3. System sends the full conversation history (as a messages array) along with a system prompt defining the AI's academic advisor persona to the OpenAI GPT-4o-mini API. 4. The AI response is streamed back using SSE (Server-Sent Events). 5. Frontend renders the streaming text token-by-token with Markdown formatting. 6. Upon stream completion, the full assistant response is saved to the database for future context. 7. The chatbot input is cleared and ready for the next query.
Alternate Flow	A: If the OpenAI API returns an error or times out, the system displays: 'The AI assistant is temporarily unavailable. Please try again in a moment.' B: If the student selects Bangla mode before sending, the system prompt instructs the AI to respond in Bangla.
Postcondition	The conversation is updated with both the student's message and the AI's response. Context is retained for follow-up queries.

5.4 Use Case: AI Quiz Generation

Field	Detail
Use Case ID	UC-004
Use Case Name	AI Quiz Generation from Notes
Actor	Authenticated Student
Precondition	The student is logged in and has at least one note uploaded.
Trigger	Student selects a note and clicks Generate Quiz.
Main Flow	1. Student navigates to the Quiz section. 2. Student selects a note from their uploaded notes list. 3. Student clicks Generate Quiz. 4. System retrieves the extracted text content of the selected note. 5. System sends the text to the OpenAI API with a structured JSON prompt requesting MCQ and short-answer questions. 6. AI returns a structured JSON array of questions, options, and correct answers. 7. System parses and validates the JSON response. 8. Quiz is rendered as an interactive question card interface. 9. Upon submission, system scores the quiz, displays results, and saves the score and topic to the database.
Alternate Flow	A: If the note text is too short (< 100 words), the system displays: 'This note does not have enough content to generate a quiz. Please upload a more detailed note.' B: If the AI response is malformed or not parseable as valid JSON, the system retries once with a corrective prompt before displaying a fallback error.
Postcondition	A quiz is generated and completed. The score is stored and will influence the Weak Area Detector and Exam Readiness Score.

5.5 Use Case: AI Study Planner

Field	Detail
Use Case ID	UC-005
Use Case Name	AI Study Planner — Schedule Generation
Actor	Authenticated Student
Precondition	The student is logged in. The student has at least one upcoming task or exam date entered.
Trigger	Student provides study inputs and clicks Generate Plan.
Main Flow	1. Student navigates to the Study Planner. 2. Student enters: list of subjects, daily available study hours, and exam dates for each subject. 3. Student clicks Generate Plan. 4. System constructs a structured prompt with the student's inputs. 5. AI returns a weekly timetable in JSON format prioritizing high-risk subjects. 6. System renders the timetable as a weekly grid with day columns and subject blocks. 7. Student can print or save the timetable.
Alternate Flow	A: If the student provides conflicting inputs (e.g., 0 daily hours available), the system displays a validation error before sending to the AI.
Postcondition	A personalized weekly study plan is displayed. The student can reference it throughout the week.

5.6 Use Case: Admin — View and Manage Users

Field	Detail
Use Case ID	UC-006
Use Case Name	Admin: User Management
Actor	Admin
Precondition	The admin is logged in. The JWT token carries the role Admin.
Trigger	Admin navigates to the Admin Panel.
Main Flow	1. Admin clicks Admin Panel in the navigation. 2. System verifies the JWT role claim is Admin. If not, returns HTTP 403. 3. System fetches a paginated list of all users from the database. 4. Admin can view each user's name, email, registration date, and last active timestamp. 5. Admin can click Deactivate on any student account, which sets the user's status to inactive and blocks login. 6. Admin can click Delete to permanently remove a student account and all associated data after a confirmation dialog.
Alternate Flow	A: If a non-admin attempts to access the admin panel URL, the system returns HTTP 403 Forbidden and redirects to the student dashboard.
Postcondition	The selected user account is deactivated or deleted. Platform statistics are updated accordingly.

5.7 Use Case: Pomodoro Focus Timer

Field	Detail
Use Case ID	UC-007
Use Case Name	Pomodoro Focus Timer with AI Recommendation
Actor	Authenticated Student
Precondition	The student is logged in and has navigated to the Timer page.
Trigger	Student enters a topic and requests a session recommendation.
Main Flow	1. Student enters the topic they intend to study. 2. Student clicks Get AI Recommendation. 3. System sends the topic to the AI with a prompt asking for an optimal Pomodoro session length. 4. AI returns a recommended duration (e.g., 25 min for revision, 50 min for new topics) with a brief rationale. 5. System pre-fills the timer with the recommended duration. 6. Student clicks Start to begin the countdown. 7. Timer counts down and plays an alert sound when complete. 8. System logs the completed session (topic, duration, timestamp) to the database.
Alternate Flow	A: Student may manually override the AI-recommended duration before starting.
Postcondition	The focus session is logged. Study hours are updated in the analytics dashboard and the student's study streak.

5.8 Use Case: View Analytics Dashboard

Field	Detail
Use Case ID	UC-008
Use Case Name	View Personal Analytics Dashboard
Actor	Authenticated Student
Precondition	The student is logged in and has been using the platform for at least one session.
Trigger	Student navigates to the Analytics page.
Main Flow	1. Student clicks Analytics in the navigation. 2. System queries the database for the student's activity logs: completed sessions, quiz scores, topic history. 3. System computes aggregate metrics: daily study hours (last 30 days), quiz performance per topic, topic coverage. 4. System renders three Recharts visualizations: a line chart for study hours, a bar chart for quiz trends, and a pie chart for topic coverage. 5. System also renders the Weak Area panel listing topics below the 60% quiz score threshold. 6. Student can hover over chart points for detailed tooltips.
Alternate Flow	A: If the student has insufficient data (e.g., no quizzes taken yet), the system displays placeholder charts with a message: 'Complete some quizzes to see your performance analytics.'
Postcondition	The student has a clear visual overview of their academic performance and areas requiring improvement.